

And we can use named parameters for clarity:

```
@Query("select u from User u where u.firstname LIKE :firstname")
List<User> findUsersWithNamesLike(@Param("firstname") String name);
```

The above examples only allow us to query data but not modify it. We can do that by using the `@Modifying` annotation.

```
@Modifying
@Query("update User u set u.firstname = ?1")
int changeFirstName(@Param("firstname") String name);
```

2. **By creating an implementation** of the repository for more control, especially when we need to perform complex operations. We can create custom implementations of repositories for finer control. To achieve this, we'll have to create a separate interface that has our custom methods declared, and we'll have to implement it and have our repository extend both `JpaRepository` and our custom repository.

```
interface CustomizedUserRepository {
    void doSomethingTricky(User user);
}
...
class CustomizedUserRepositoryImpl implements
CustomizedUserRepository {

    public void doSomethingTricky(User user) {
        // Custom implementations go here
    }
}
```

As we have our custom interface and implementation ready, we can now attach it to our actual user repository by utilising Java interface's multiple inheritance property:

```
@Repository
interface UserRepository extends JpaRepository<User,
Long>, CustomizedUserRepository {
    // ...
}
```

Now we can use it in a service in the way we need to.

There are more features to Spring data repositories like transactions, sorting, projections, named queries, and stored procedure usage, etc, that can be found at <https://docs.spring.io/spring-data/jpa/docs/2.5.1/reference/html/>.

However, to make the JPA repository work, we have to enable it from the Spring configuration with the `@EnableJpaRepositories` annotation — we can put it on the `SpringBootApplication` class, which contains the main class of the application.]

Components: Spring will handle a component, which is a generic bean. Repositories, services, and controllers are, in fact, specialised components. Components can be used for items that don't exactly fit into the categories of repositories, services, or controllers. The `@Component` annotation on a class defines it. A class that changes data from one form to another is an example. It doesn't have any explicit business implications, but it does add functionality to the program that is required by the business logic.

Configuration: Spring enables component-level tweaking and configurability to meet a variety of requirements. A database connection, for example, may require a certain IP address and port. Spring features a properties file that may be used to change the behaviour of specific beans at a global level. This can be done by modifying `src/main/resources/application.properties` in the `src/main/resources/application.properties` file. Consider the following scenario: By modifying the `server.port` property, we can change the default port that Spring listens to, or we can enable lazy initialisation of Spring components:

```
server.port=80
spring.main.lazy-initialization=true
```

We can have different application properties that can be based on the environment we're working on. We can create a separate `application-dev.properties` file and we can ask Spring to read that by adding the `spring.profiles.active` property.

```
spring.profiles.active=dev
```

Spring also allows YAML to be used for setting properties. This can be done by creating an `application.yml` file in the `src/main/resources/` directory and following the same naming scheme but in YAML:

```
spring:
  application:
    name: timeline
  datasource:
    driverClassName: com.mysql.jdbc.Driver
    url: jdbc:mysql://localhost/test
server:
  port: 80
```